MEDIKA — AI Model

Build guide, file structure & full code explanations

CS301 Project Management | Group 33 | Copperbelt University

What this module does: Takes a symptom description as plain text, runs it through a trained machine learning model, and returns one clear result: a likely disease, whether to treat at home, visit a clinic, or go to hospital, and what over-the-counter medicine to take if appropriate. The backend uses the 'action' field in the response to trigger referral booking.

1 — How the AI model works

The model has three layers that run in order every time a patient enters symptoms:

Layer 1 — Red flag check (always runs first)

Before the ML model is consulted, the input text is scanned against a list of dangerous phrases in `rules.json`. If anything like 'chest pain', 'difficulty breathing', or 'convulsions' is found, the model immediately returns action='emergency' and skips everything else. This is a hard safety rule — the ML model cannot override it.

Layer 2 — Machine learning classifier

The symptom text is fed into a trained pipeline: TF-IDF vectorisation converts the text into numbers, then a Logistic Regression classifier predicts the most likely disease out of 41 possible conditions. The model was trained on the Kaggle disease-symptom dataset with ~98% accuracy on the test set. If confidence is below 25%, the model responds with 'Uncertain' and directs the patient to a clinic.

Layer 3 — Rules-based action decision

Once a disease is predicted, `rules.json` is checked to decide what action to take. Diseases like Malaria, Tuberculosis, and Diabetes are in the 'refer_to_hospital' list. Conditions like UTI and Migraine are in 'refer_to_clinic'. Everything else gets OTC advice. Age is also checked — patients under 5 or over 65 are automatically referred to a clinic.

2 — File structure

File	Purpose	Action
`medika_ai/`	Root folder of the AI service	
`data/`	Put all 4 Kaggle CSVs here	
`dataset.csv`	4920 rows: Disease + Symptom_1..17	
`symptom_Description.csv`	Disease descriptions	
`symptom_precaution.csv`	4 precautions per disease	
`symptom_severity.csv`	Symptom severity weights 1-7	

models/	Auto-created by train.py	
model.pkl	Trained classifier (saved after train.py runs)	
disease_info.json	Descriptions + precautions per disease	
meta.json	Accuracy + training stats	
rules.json	OTC advice, refer lists, red-flag phrases	Edit this
train.py	Load data, train model, save artifacts	Run ONCE
model_core.py	Brain - diagnose() function used by both below	Never run directly
test_cli.py	Terminal tester - test without any backend	Run to test
app.py	Flask API on port 5001 - connect backend here	Run for backend
requirements.txt	Python dependencies	pip install -r

3 — Setup: step by step

Step 1 — Install Python dependencies

Open a terminal in the `medika_ai/` folder and run:

```
pip install -r requirements.txt
```

This installs: Flask, flask-cors, scikit-learn, pandas, joblib.

Step 2 — Download and place the Kaggle dataset

Go to: kaggle.com/datasets/itachi9604/disease-symptom-description-dataset. Download the dataset and place these 4 CSV files inside the `data/` folder:

- dataset.csv
- symptom_Description.csv
- symptom_precaution.csv
- symptom_severity.csv

Step 3 — Train the model

Run this once to train and save the model:

```
python train.py
```

This will print accuracy (expect 95-98%) and save `model.pkl` and support files to `models/`.

Step 4 — Test in the terminal (no backend needed)

Test the model right away without Flask or a backend:

```
python test_cli.py
```

Choose option 1 to type your own symptoms, or option 2 to run all 14 built-in test cases. Use this to verify the model is working correctly before connecting the backend.

Step 5 — Start the Flask API (when ready for backend)

When the backend developer is ready to connect:

```
python app.py
```

The AI service runs on `http://localhost:5001`. The backend should proxy calls from its own `POST /api/diagnose` route to this address.

4 — What each file does

train.py — runs once to build the model

You run this file ONCE before anything else. It reads the Kaggle dataset, trains the classifier, and saves everything the model needs to the models/ folder.

What it does, step by step:

- Reads dataset.csv and joins all symptom columns into one text string per row, e.g. 'itching skin rash nodal skin eruptions'.
- Reads symptom_Description.csv and symptom_precaution.csv to build a lookup of disease descriptions and precautions.
- Splits the data 80% for training, 20% for testing.
- Trains a TF-IDF vectoriser + Logistic Regression pipeline. TF-IDF turns words into numbers; Logistic Regression learns which number patterns correspond to which disease.
- Evaluates accuracy on the test set and prints a full classification report.
- Saves model.pkl, disease_info.json, and meta.json to models/.

model_core.py — the brain, never run directly

This is the shared engine. Both test_cli.py and app.py import this file. It loads the model once at startup and exposes one function: diagnose().

The diagnose() function:

```
result = model_core.diagnose('fever headache skin rash')

# result is a dict:
# {
#   'disease':     'Malaria',
#   'confidence':  0.91,
#   'action':      'hospital',   <-- backend reads this
#   'advice':      'This may be Malaria. Go to hospital now.',
#   'description': '...',
#   'precautions': ['Consult doctor', 'Use mosquito nets'],
#   'refer':       True,
#   'disclaimer':  '...'
# }
```

The action field is the key output for the backend:

action value	Label	What backend does
otc	Home treatment	Minor condition. Show OTC advice. No referral needed.
clinic	Visit clinic	Moderate condition. Backend books a clinic appointment.
hospital	Go to hospital	Serious disease. Backend creates a hospital referral.
emergency	Emergency	Red-flag symptom detected. Backend shows nearest hospital immediately.

rules.json — the rules you can edit

This file controls referral decisions and OTC advice. You do not need to retrain the model to change it — just edit the file and restart app.py.

Three sections:

- : List of disease names that should always trigger a hospital referral. Add or remove diseases freely.
- : Diseases that need a clinic visit but not a hospital.
- : Plain-English advice for each disease that can be managed at home. Includes medicine names.
- : Phrases that trigger the emergency response before the ML model even runs.

test_cli.py — test without a backend

Run this any time to check the model is working. It has two modes: type your own symptoms, or run the full built-in test suite.

The test suite covers all four action types:

- OTC cases: Fungal infection, Common cold, Allergy, Chicken pox
- Clinic cases: UTI, Migraine, Drug Reaction
- Hospital cases: Malaria, Typhoid, Tuberculosis, Jaundice
- Emergency cases: chest pain, convulsions
- Edge case: vague input with low confidence

app.py — Flask API, connect backend here

This is the only file the backend developer needs to know about. It wraps model_core.py in a simple HTTP API running on port 5001.

Endpoints:

```
POST http://localhost:5001/diagnose

Request body:
  { "symptoms": "fever headache rash" }

Response:
  { "disease": "Malaria", "action": "hospital", ... }

---

GET http://localhost:5001/health

Response:
  { "status": "ok", "accuracy": 0.97, "n_diseases": 41 }
```

How the backend connects (backend/routes/diagnose.py):

```
import requests

AI_URL = 'http://localhost:5001'   # set in config.py

@app.route('/api/diagnose', methods=['POST'])
def diagnose():
    symptoms = request.json.get('symptoms')
    res = requests.post(f'{AI_URL}/diagnose',
                        json={'symptoms': symptoms},
                        timeout=10)
    result = res.json()

    # Use action field to decide what to do
    if result['action'] == 'hospital':
        create_hospital_referral(current_user, result['disease'])
    elif result['action'] == 'clinic':
        create_clinic_appointment(current_user, result['disease'])

    return jsonify(result)
```

5 — Quick reference

Order to run files

1. python train.py — run once after placing CSV files in data/
2. python test_cli.py — test the model in the terminal
3. python app.py — start the Flask API when backend is ready

Common errors and fixes

FileNotFoundError: models/model.pkl not found → Run python train.py first.

FileNotFoundError: data/dataset.csv not found → Download the 4 CSVs from Kaggle and place them in data/.

UnicodeDecodeError reading CSV → Already handled. train.py tries UTF-8 first, then latin-1.

Port 5001 already in use → Kill the existing process or change port in app.py.